Contributions to Open-Source GIS Projects

Free Software GIS – Semester Report
Department of Geomatics
Faculty of Civil Engineering

Czech Technical University in Prague

Mykhailo Radchenko

June 2026

1 Introduction

This report summarizes the work completed during the semester within the Free Software GIS course. The main goal of the work was to contribute to real open-source geospatial software projects and to document both the technical implementation and the practical impact of the submitted changes.

The work focused on contributions to the GRASS and QGIS ecosystems. Instead of creating an isolated semester project, I worked directly with production repositories, existing codebases, and standard open-source contribution workflows including issue analysis, implementation, testing, code review, and iterative revision of pull requests.

A total of five pull requests were completed during this semester. These contributions addressed different types of problems, including software compatibility, authentication workflows, graphical user interface improvements, and export of styling logic.

In addition to the implementation itself, the work also provided practical experience with upstream collaboration, continuous integration requirements, and the need to adapt code based on maintainer feedback. This was especially valuable because some contributions required not only new functionality, but also writing or updating automated tests to satisfy project quality standards.

Table 1 provides a summary of all pull requests submitted during this semester.

Table 1: Overview of submitted pull requests and their current status.

Project	**Topic**	**Pull Request**	**Status**
GRASS GIS Addons	EODAG v3 and v4 compatibility	#1663	Merged
GRASS GIS Addons	Automated MFA for Creodias	#1685	Merged
GRASS GIS	Copy and paste of module parameters	#7237	Merged
GRASS GIS	GUI refactoring and persistence	#7404	Merged
QGIS	Concatenation in SLD export	#65366	Merged

2 Work Methodology

The work was carried out using a standard open-source development workflow. Each contribution started with analysis of the problem, followed by implementation, local testing, interaction with maintainers, and refinement based on review comments.

An important part of the process was adapting the work to the expectations of the target projects. In several cases, it was not sufficient to only implement a new feature or fix; maintainers also required corresponding test coverage or updates to existing tests so that the changes could be verified in continuous integration pipelines.

The individual tasks differed in scope. Some contributions focused on backend logic and compatibility with external libraries, while others improved user interaction in desktop GIS software. This combination provided experience with both low-level technical debugging and user-facing design considerations.

Another useful aspect of the semester was the opportunity to further develop practical skills in AI-assisted development. AI agents were helpful mainly as support tools for exploring possible approaches, speeding up orientation in unfamiliar parts of the codebase, and refining implementation ideas, while the final integration, testing, and project-specific adjustments still required manual work and understanding of the code.

3 PR 1: GRASS Addons – EODAG v3 and v4 compatibility

Pull Request: OSGeo/grass-addons#1663

3.1 Context

This contribution focused on the `i.eodag` module in GRASS Addons. The task was motivated by changes introduced in EODAG v4, which affected the compatibility of the module and required updates to both the implementation and the testing workflow.

3.2 Objective

The objective of this pull request was to add support for EODAG v4 while preserving backward compatibility with EODAG v3. This was important in order to keep the addon usable across different environments and dependency versions.

3.3 Implementation

In this part of the work, I analyzed the API differences between EODAG v3 and v4 and updated the module logic accordingly. The implementation had to handle version-specific differences in returned objects and internal behavior while keeping the user-facing functionality stable. In practice, this meant ensuring compatibility not only in the main script itself, but also in the related test script used by the project.

3.4 Testing

An important part of the contribution was updating the automated tests so that the new logic could be reliably verified in continuous integration. The testing workflow had to reflect the same compatibility goals as the implementation itself, namely supporting both EODAG v3 and v4 and confirming that the module remained functional in representative data discovery and processing scenarios.

3.5 Result

The result of this contribution was improved robustness of the `i.eodag` addon with respect to upstream library evolution. This is important because the module depends on an actively developed external project, and long-term usability requires maintaining compatibility not only in the production code but also in the automated test infrastructure.

4 PR 2: GRASS Addons – automated MFA for Creodias

Pull Request: OSGeo/grass-addons#1685

4.1 Context

The second contribution also targeted the `i.eodag` addon, but this time the focus was on authentication. The practical problem was that access to the Creodias provider required multi-factor authentication, which complicated automated workflows and reduced usability.

4.2 Objective

The goal was to make authentication more user-friendly and more suitable for scripted usage by reducing manual intervention during the login and download process.

4.3 Implementation

The implemented solution introduced automated handling of one-time passwords. This required integration of TOTP-based logic together with careful handling of timing-related issues so that generated codes were accepted reliably. The implementation was designed to make the authentication workflow smoother for end users while keeping the process fully functional with the Creodias provider.

4.4 Testing

Unlike some of the other contributions, this pull request was not accompanied by automated CI tests. Instead, the functionality was validated manually using my own Creodias credentials and real authentication attempts against the provider. This manual testing was necessary because the feature depended on an external authentication workflow and real-time generation of valid OTP codes.

4.5 Technical Challenges

A significant technical aspect of this work was that authentication is sensitive to time synchronization. Because of that, the implementation had to consider not only generation of OTP tokens but also practical reliability of the whole workflow. This was particularly important for ensuring that automatically generated codes were accepted consistently in real usage.

4.6 Result

This contribution improved the usability of the addon for users of the Creodias provider. From the perspective of the course, it is a strong example of a change that is both technically nontrivial and directly beneficial in real data-access workflows. It also provided valuable practical experience with authentication mechanisms that are difficult to test in a purely isolated way.

5 PR 3: GRASS – copy and paste of module parameters

Pull Request: OSGeo/grass#7237

5.1 Context

This contribution targeted the main GRASS repository and focused on the graphical user interface. The problem addressed here was that transferring module parameters between dialogs was not convenient, especially in repetitive workflows.

5.2 Objective

The objective was to improve efficiency and usability of module dialogs by adding mechanisms for copying and reusing parameter settings.

5.3 Implementation

The implemented feature added support for copying module configuration from the dialog and for pasting parameters back into another dialog instance. This type of feature improves user productivity, especially in workflows where similar processing steps are repeated many times with small variations.

An important part of this work was learning to work with GUI design in wxPython, which was a less familiar area for me than backend-oriented development. The implementation therefore required not only changes in application logic, but also attention to widget placement, interaction design, and the overall usability of the dialog.

5.4 Development Process

This contribution also showed that GUI development is often iterative. Some intermediate versions were mainly useful for proving that the feature worked, while later revisions were needed to make the interface more natural and visually consistent.

5.5 User Impact

Although the implementation was less focused on algorithms than some of the other PRs, it had clear practical value. GUI improvements are important because even small usability enhancements can significantly improve daily work with GIS software.

5.6 Result

The main result was a more convenient and efficient dialog workflow in GRASS. This contribution also required attention to interface design, not only to internal code behavior, and it provided valuable experience with practical GUI development in wxPython.

6 PR 4: GRASS – follow-up GUI refactoring and persistence

Pull Request: OSGeo/grass#7404

6.1 Context

After the initial GUI enhancement was merged, a maintainer opened a follow-up issue requesting further UX improvements and better integration with the global application settings. This pull request was prepared directly in response to that feedback to refine the design and improve the consistency of the solution.

6.2 Objective

The goal of this PR was not to introduce a completely new tool, but to strengthen and polish the previously added functionality based on the maintainer's review. This included code-level optimizations and proper integration of the feature into the existing GUI settings architecture.

6.3 Implementation

The work involved refactoring the copy-related logic using `StringIO` for more efficient string handling and unifying the Python API selection within the global GUI settings. Additionally, the implementation introduced state persistence, ensuring that the user's preferred copy format is stored and remembered across different GRASS sessions. This transformed the original feature from a useful but isolated addition into a fully polished and maintainable component.

6.4 Result

This PR highlights an important aspect of open-source development: the work often does not end with the first merged pull request. Responding to maintainer issues, performing follow-up refactoring, and ensuring consistency with project conventions are essential parts of making high-quality contributions.

7 PR 5: QGIS – concatenation support in SLD export

Pull Request: qgis/QGIS#65366

7.1 Context

The final contribution was made to the QGIS project. It addressed export of label styles to SLD, specifically support for expressions involving string concatenation.

7.2 Objective

The objective was to improve correctness of style export so that label expressions using concatenation are represented properly in the exported SLD output.

7.3 Implementation

This required changes in the logic responsible for converting internal expression structures into an OGC-compatible export representation. The task therefore involved understanding both the QGIS expression system and the constraints of the target interchange format. In practice, the implementation focused on extending the export logic so that concatenation expressions were translated correctly instead of being lost or exported incorrectly.

7.4 Testing

An important part of this contribution was verifying that the new export behavior worked correctly in an automated way. To support continuous integration, it was necessary to add tests covering label expressions with string concatenation and their expected SLD output. This made the change easier to validate and helped ensure that the new behavior would remain stable in future development.

7.5 Importance

This contribution is a good example of interoperability work. Even relatively small changes in export logic can be important because they affect portability of styles between systems and compliance with open standards.

7.6 Result

The result was improved support for a real expression pattern used in labeling workflows. This makes the export functionality more complete and more reliable for practical use. It also provided valuable experience with contributing not only implementation code, but also test coverage required for integration into a large upstream project.

8 Discussion

The five pull requests covered several different aspects of software engineering in GIS. They included dependency compatibility, authentication, desktop GUI usability, code refactoring, automated testing, and standards-based export.

From a broader perspective, this diversity was valuable because it required switching between different ways of thinking. Some tasks were primarily about library integration and backend behavior, while others required attention to user experience, software design consistency, and interoperability.

Another important aspect of the work was collaboration with existing projects. Contributing to established repositories means following coding conventions, responding to review comments, and adapting code until it is acceptable for inclusion in the upstream project. In some cases, this also meant writing or updating tests required by continuous integration workflows, which was a valuable part of the development experience.

The semester was also useful in terms of development workflow and tooling. Besides programming itself, I gained more experience with AI-assisted development as a supporting tool for exploring possible solutions, understanding unfamiliar parts of the codebase, and accelerating iteration, while the final implementation, debugging, and adaptation to project requirements still required manual work and technical understanding.

9 Conclusion

The semester work demonstrated that open-source contribution can serve as a meaningful and academically valid form of project output. Instead of producing only a local prototype, the work resulted in changes integrated into widely used open-source GIS projects.

The contributions brought practical improvements to GRASS Addons, GRASS, and QGIS. At the same time, the work provided valuable experience with real development workflows, code review, testing, continuous integration requirements, and maintenance of production-quality code.

From the perspective of the Free Software GIS course, the project was beneficial in two ways: it produced concrete results for the open-source GIS community, and it significantly deepened my technical skills in geospatial software development. It also strengthened my ability to work with larger codebases, respond to maintainer feedback, and contribute changes that meet upstream quality expectations.